

String Operations

Learning Objectives

- Manipulate strings using concatenation, slicing, and common methods
- Apply string functions to solve text processing problems
- Understand how strings are stored as character arrays internally

Engage (5-7 minutes)

Every time you type a message, search the web, or fill out a form, you work with strings. A programmer needs to manipulate text—extracting a username from an email, validating a phone number format, or converting names to uppercase. Strings are one of the most frequently used data types, and mastering their operations is essential. Today we explore powerful string manipulation techniques.

Explore (10-12 minutes)

Take the string “Hello, World!” and experiment: extract “World” using slicing, convert it to lowercase, count the number of characters, and replace “World” with “Students”. Then try concatenating your first name and last name with a space. What happens when you try to change a single character directly—like changing ‘H’ to ‘J’?

Explain (10-12 minutes)

Strings are sequences of characters stored as character arrays in memory. Common operations include concatenation (joining strings), slicing (extracting substrings), and methods like `upper()`, `lower()`, `split()`, `join()`, `replace()`, and `strip()`. Strings are typically immutable in many languages, meaning you cannot change individual characters directly—instead, you create a new string. Understanding string indexing and slicing patterns is crucial for text processing tasks like parsing data and validating input.

Elaborate (8-10 minutes)

Write a program that takes a full name as input and outputs initials (e.g., “Rahul Kumar Sharma” becomes “R.K.S.”). Then build a function that checks if a given string is a palindrome. Create another function that counts the frequency of each character in a sentence. Finally, write code to extract all email addresses from a given text using string methods.

Evaluate (5-8 minutes)

Given the string " Python Programming ", write the output of: `strip()`, `upper()`, `replace("Python", "Java")`, `split()`, and `len()`. Then write a function that capitalizes the first letter of every word in a sentence without using the built-in `title()` method.